Human-AI Collaboration Design

Strength of each player

<u>Human</u>

- Have emotions and lived experiences
    - However, many humans may not be particularly good at directly typing their emotions and lived experiences when asked to
- Humans can express their emotions through music based on their understanding and feeling form lyrics, rhythm, melody, and tempo
- Humans are good at talking about their stories through speech and expressing their emotions through tone.
- Humans are good at using memes/emojis to convey emotions.
- Humans know if something is beautiful, and how different styles of art give different feelings.

<u>AI</u>

- Can process different forms of emotional input [music, emojis, etc] quickly to efficiently understand the user's feelings
- Create sketches fast from basic boilerplate
- Quick iterations, and update the code with changes
- Debug and fix any code technical problems quickly

<u>Other Algo</u>

- Code validation
- Code execution and rendering sketches
- Enforce the constraints that are set up

Based on the strengths of each player, we designed the responsibilities/tasks for each player.

Division of Responsibility

<u>Human - novice programmers</u>

- The human have the responsibility of deciding the message that the sketch is conveying, they fulfil this responsibility by:
    - Initiating the conversation of the team, answering and clarifying questions asked by the AI to specify their goals
    - Explain their emotions using many formats which feel natural to them. They can write text. They can share images, such as memes or photos. They can upload

music or audio. They can use emojis. They can even provide rough sketches or simple prototypes.

- The human has the responsibility of artistic judgment or direction. They fulfil this by
    - Giving certain directions for the artistic style of this sketch to the AI
    - Plus, after each iteration, the user gives feedback to the AI. They say what feels right and what does not. They might decide the motion is too fast, the colour feels wrong, or the scene does not feel lonely enough. They decide whether to keep the change, adjust it, or try something new. This keeps them in control of the creative process.

<u>AI</u>

- The AI has the responsibility of helping the human to better define or express the lived experience they have in mind. The AI fulfils this responsibility by asking structured, clarifying or guiding questions, including prompting users for additional input, such as images, music, audio, etc that may make it better to express
- The AI has the responsibility of technical implementation of the code that follows the lived experience or emotion defined. They fulfil this responsibility by
    - Converting the ideas developed in earlier conversations into implementable p5.js logic
    - Writing syntactically correct code, including clear and detailed comments so the human can understand what each part does.
    - Providing parameters that the human can adjust without too much knowledge in the code syntax. For example, the human might want to change particle count, speed, or colour intensity.
    - Optionally edits specific steps in the code history or provides different versions of the sketch for the human to compare. (optional)

Autonomy

<u>Human</u>

- The human retains creative and intentional autonomy by deciding the following:
    - Emotional goal or lived experience to express
    - Medium of expression: either text, emojis, images, rough sketches, or audio
    - The aesthetic direction of the output
    - Ability to accept, modify, or discard generated iterations
- The system is designed so that the user owns the meaning of the artwork that will be outputted which preserves creative control and prevents automation of self-expression

<u>AI</u>

- Has technical and certain stylistic autonomy by autonomously:

    - Interprets multimodal emotional input
    - Proposes visual metaphors or stylistic approaches
    - Structures p5.js logic
    - Determines efficient and syntactically correct code organization
    - Suggests tweakable parameters

Shared context and mutual awareness

The shared context in this project emerges gradually through interaction. Initially, the human holds internal emotional knowledge, lived experience, and aesthetic intention while the AI lacks that internal context. Through structured clarifying questions, the AI incrementally builds a representation of:

- Emotional tone
- Narrative setting
- Desired pacing and motion
- Visual style
- Symbolic associations

By allowing multimodal input (music, emojis, memes, audio, sketches), the system broadens the shared context beyond text. This reduces ambiguity and increases alignment between intention and output. The mutual awareness develops as:

- The AI understands what emotional signal the user wants to convey
- The user understands how the AI translates emotion into code and visual structure
- Both the user and AI communicate on the shared chat with iterations so the AI can better understand the user's needs and the user can provide feedback

Communication and coordination mechanism

<u>Human → AI Communication</u>

The human communications through natural language describing their feelings and stories, emojis to depict emotional signals concisely, music or audio, or sketches. This flexibility acknowledges that novice programmers may struggle to formally describe the emotions in a language optimized for the AI. Instead of forcing structured input, the system supports expressive input. Additionally, this allows for additional iterating and more communication between the human and AI.

<u>AI → Human Communication</u>

The AI communicates through structured clarifying questions in natural language, transparent reasoning steps, well-commented p5.js code, parameterized controls, and alternative design

versions. Rather than simply outputting code, the AI exposes its translation logic supporting both trust and learning.

Coordination Structure

Follows an iterative cycle:

1. User expresses intention
2. AI asks clarifying and guiding questions
3. AI generates initial implementation
4. Supporting algorithm validates and renders
5. User provides feedback
6. AI adjusts selectively

Limiting how much code changes per iteration prevents overwhelming shifts and keeps coordination stable. The “option to edit, save, undo” adds control checkpoints, further stabilizing collaboration. This structured turn-taking creates predictable coordination patterns, reducing cognitive overload for novice users.

Mutual adaptation and team learning

Human adaptation

As users interact with the system, they:

- Learn how certain emotional descriptions map to visual effects
- Learn how emotions or lived experiences can be better expressed or defined
- Gain familiarity with p5.js structures
- Develop intuition about parameters (speed, color, randomness)
- Gradually shift from passive requester to active modifier

The optional Learning Mode toggle formalizes this adaptation. When enabled:

- The AI explains the code logic more deeply
- Comments become more instructional
- The user can experiment with parameters intentionally

AI Adaptation

Within a single session, the AI refines its understanding of the human’s aesthetic and emotional expectations. Adapting through:

- Incorporating the human’s feedback after each iteration
- Narrowing stylistic options based on the human’s preferences
- Adjusting visual metaphors based on corrections

Across sessions, the AI can gain more knowledge by having more examples for fine-tuning or including a few-shot example in the system instructions. It adapts in the following way.

- Gain more knowledge on ways that may better help humans understand, define or express emotions or lived experiences by seeing what medium, structure/tone/langugage of clarifying questions works the best
- Gain more knowledge on best practices in code by absorbing feedback on code or correcting bugs
- Gain more knowledge on stylistic options based on feedback and preferences

<u>System-Level Learning</u>
The team users and AI demonstrate adaptation as follows:

- Improved alignment between emotional input and visual output
- Reduced clarification rounds over time
- More precise parameter suggestions
- Increased human confidence

Interaction Design

Target Users: Novice programmers

Shared goal: create p5.js sketches that creatively communicate a unique life experience

Solutions
Learning mode toggle: users can choose whether to learn coding. If yes, there will be more autonomy for the user in the code

Users will initiate the conversation.
AI will follow up with clarifying questions:

- Text
- Talk about your situation and your emotions. What is the story, what is the setting, how do you feel? [Audio input]
- Is there a song that can capture your emotion?
- What are the three memes/emojis that best describe your emotion?
- Upload an image that shows your emotion, the style you want…
- If you have a sketch, feel free to upload it.

After specifying, the AI model generates codes for several iterations

- Take transparent steps when generating
- Limit the number of new lines added to the existing graph before proceeding to make sure the direction is correct, and the process is transparent
- Provide tweakable parameters

Other algo (a compiler, renderer) shows a preview of the photos/videos that the code generates

- Option to edit, save, and undo

Users will (supported by other algo)

- Choose one iteration, insert code into the user's codespace, and modify based on it
- Regenerate the code for a new idea

Code will include detailed comments for each line/chunk of code

- Could select certain codes to change (e.g., colors, shapes…) ask AI to modify the code and replace the new code in

Research Notes

Machine Learning

Motivations

When assessing current AI models, we first researched if there were existing AI models for p5.js and found p5.js AI Studio. Our evaluation of this tool motivated the direction of our AI framework.

Limitations of the current p5.js AI Studio product

- We noticed that their AI Studio only allowed text input, which limits the depth of emotional data that the LLM can use. Most people are more comfortable describing events in text descriptively rather than emotionally, making it hard for LLMs to contextualise emotion without making general assumptions of how they may have felt. Additionally, even if the LLM was to explicitly ask "how do you feel about this event?," some users may feel uncomfortable stating an exact emotion or may even have trouble defining more complex feelings.
- People often find it easier to express their emotions through alternative outlets than text, so we want to leverage these outlets as a way to gather more contextual input from our user. Some of these alternatives include auditory and visual inputs: for example, with auditory narrations, the LLM can pick up tone, pitch, or expressiveness in the user's voice. It may even be able to more easily detect additional nuances such as sarcasm in speech. Furthermore, if users provided images or emojis to express their feelings, it may be more exact than written responses about emotions.
- After asking the agent to make a sketch of some event, it seems to immediately generate the code with little to no followup. For example, when I asked it to generate a sketch to represent my "first day of classes," the chatbot immediately generated a sketch that embodied "nervousness." This suggests that the studio makes general assumptions about a typical student's first day of class, but it may not accurately reflect my actual emotions on that day, which were more multifaceted with excitement, bittersweetness, and nervousness.

Therefore, there is a need for our LLM to deeply understand the user's emotion(s) about a particular event before outputting code. We want our chatbot to converse with the user to receive as much emotional context as possible, preventing it from making general emotional assumptions about an experience. We decided that prompt engineering would best fulfill this goal.

Because we are delegating responsibility for the LLM to generate sketch code, we also need to specialize a baseline LLM for the task of generating image code.

ML Data Flow

Input: Multimodal
- Natural language prompt
- Optional image
- Optional audio

Output:
- Natural language follow-up questions or explanations
- Code with natural language explanation

(Additional system instructions/prompt engineering and finetuning datasets, see below)

ML Framework

Existing off-the-shelf AI Model: Gemini (2.5 Flash primarily)
- We researched state-of-the-art baseline LLMs (e.g., Claude, ChatGPT, Gemini) because they are generically helpful for all tasks and have standardised APIs or platforms that make them easier to fine-tune for specific tasks or prompt engineering.
- We ultimately chose Gemini because:
    - It allows for multimodal (code, text, audio, image, etc.) input and output (https://developers.googleblog.com/en/7-examples-of-geminis-multimodal-capabilities-in-action/)
    - It has strong capabilities for coding, ranking high in benchmarks, even for the flash version (https://livebench.ai/#/?sort=Coding+Average)
    - Its API has a free-to-use limit (https://ai.google.dev/gemini-api/docs/pricing) and fine-tuning it on Google Cloud has free $300 credits and a trial period (https://docs.cloud.google.com/free/docs/free-cloud-features)

Fine tuning

Because the LLM's primary task is to translate emotions to image code, we wanted to optimise Gemini's ability to translate both text labels and images to code.

Datasets for fine-tuning - for most of these, we will need to web scrape
- Fine-tuning for creativity
    - Websight Dataset: https://huggingface.co/blog/websight. The dataset has training data of screenshots to HTML code, which would help the model better understand how to translate image inputs to code. Very large, more than 823,000 examples.

        - Interactive Diagrams to Code on Open Processing: https://openprocessing.org/browse?q=&time=thisMonth&type=tags&offset=360#. Very large dataset, over a million in total.
- Fine-tuning for specific expertise in p5.js.
    - Code best practices and syntax-specific data
        - p5.js reference: https://p5js.org/reference/
        - p5.js tutorial: https://p5js.org/tutorials/
        - Stackoverflow threads with p5.js tags: https://stackoverflow.com/questions/tagged/p5.js?tab=Votes. Relatively more data, but the quality varies a lot.
    - Image/text to interactive p5.js code examples (many p5.js code snippets and their corresponding graphics/sketches rendered). In total, we found around 400+ examples.
        - Examples on p5.js official docs: https://p5js.org/examples/. Around 60 examples. High-quality sketches and good coding practices are from the official documentation.
        - Community works on p5.js: https://p5js.org/sketches/. Around 50 examples. Very high-quality docs.
        - Examples on p5.js AI gallery https://p5js.ai/gallery/. 100+ examples, but quality varies a lot.
        - p5.js AI code guides: https://p5js.ai/sketchdocs. Around 200 examples, but the quality varies a lot.
- Fine-tuning for emotions or lived experiences
    - May have to manually create examples of prompts with preferred outputs to feed into Preference-based fine-tuning (see below)

How to fine-tune: read the official tutorial of Google Cloud for tuning https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/tune-models

- On Google Cloud Vertex AI
- Supervised fine-tuning - fine-tune with labelled data
    - Collect datasets, upload as JSON format
    - Focus on reading finetuning with image (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/tune_gemini/image_tune) and audio (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/tune_gemini/audio_tune)
- Preference fine-tuning - can be an important strategy for fine-tuning to catch user emotions and lived experience
    - Let's tune with human feedback data - learn from subjective preferences that are hard to define by using specific labels or through supervised fine-tuning alone

    - The input dataset contains examples consisting of a prompt and a pair of responses, indicating which one is preferred and which one is dispreferred. The model learns to generate preferred responses with higher probability and dispreferred responses with lower probability.
    - Tutorial/overview: https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini-preference-tuning

Pricing: https://cloud.google.com/vertex-ai/generative-ai/pricing

- For Gemini 2.5 Flash:
    - $0.30 per 1M text/image input tokens,
    - $1 per 1M audio input tokens
    - $2.50 per 1M text output tokens (we should only need text output. In fact, image output is very expensive)
- Need to be careful not to exceed 300 free credits

Prompt Engineering

In addition to fine-tuning, we also aim to customise Gemini by prompt engineering.

Specifically, we will design a prompt engineering framework that allows our LLM to carefully ask the user to express their emotions about an event in different inputs. The purpose of this is to optimise emotional understanding so the LLM can closely visualise what the user is feeling. The chatbot should assess the user's initial input for emotional specificity and follow up with the user for more input formats if needed [music, emojis, etc] so the LLM gains more emotional context.

Also, as part of prompt engineering, we want to put in place certain guardrails. For example, cautioning sketches that may be unethical or illegal.

Tutorials for how to engineer prompts: https://docs.cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/introduction-prompt-design

- Should focus on reading on techniques for designing multimodal prompts on Gemini official docs: https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/design-multimodal-prompts

Design

Product: LLM-powered chatbot for co-creating expressive p5.js sketches

Users: Novice programmers

Shared Goal: Create a p5.js sketch that creatively communicates a unique life experience

Problem Definition

People often struggle to express personal emotions and create art through code because they face two simultaneous challenges: technical barriers in coding (syntax, structure, debugging) and conceptual barriers (translating abstract emotions into visual systems). Traditional AI coding assistants risk over-automation, where users receive a well-drawn artwork (say from ChatGPT) without understanding or ownership. Our design challenge is therefore:

How might we design an AI system that enhances creative expression while preserving user agency, learning, and ownership?

The product we design should be human-centred and emphasise collaboration between humans and AI rather than replacement.

User Research

Through observing novice programmers and reflecting on learning experiences, we identified three core pain points:

Insight 1:
Novices often spend disproportionate time debugging small errors or recalling syntax, which interrupts creative flow.

Insight 2:
Users can describe feelings vividly in natural language but struggle to map those emotions into parameters such as motion, color, and density.

Insight 3:
When AI produces complete outputs, users feel less connected to the final artifact and report lower satisfaction and learning. There is not enough transparency or flexibility for users. It is also hard to keep outputs consistent across iterations (preventing the model from drifting styles or rewriting everything)

Design Principles

From these insights above, we derived two core principles:

1. The human provides lived experience and evaluates emotional authenticity. The AI converts abstract emotion into concrete visual logic.
2. The system must require user decisions before generating code and limit each iteration to incremental changes to ensure transparency.

Evaluation Strategy

We evaluate the product across:

Objective Performance

- Does the sketch run correctly?
- Does iteration converge toward user intention?

Collaboration Quality

- Did user make creative decisions?
- Was AI contribution meaningful but bounded?

Intelligibility

- Could users predict visual changes?
- Did they understand what was modified?

Trust

- Did users feel in control?
- Did they rely appropriately on AI?
- Did recovery mechanisms work?

- Subjective Teamwork Experience
- Did users describe the AI as a partner rather than a tool?
- Did they feel ownership over the final artifact?

App Dev

We researched existing tools and frameworks we could use to build our LLM powered chat bots, with the goal of finding a stack that can support real time p5.js rendering, multimodal input, and iterative AI collaboration.

Technical skills:

- JavaScript (including browser-based applications)
- Basic frontend frameworks (React)
- p5.js creative coding
- REST APIs
- Basic backend (Node.js / Flask)
- API integration (OpenAI, Anthropic, Gemini)
- Prompt engineering for structured outputs

Architecture:

- Frontend: React for frontend and p5.js for sketch rendering
    - React's component based architecture maps cleanly onto our UI elements and will allow for fast development using common React libraries
    - We can also utilize React's state management to save chat history, current code, and canvas rendering
    - For p5.js integration within React, we will use the react-p5 wrapper library (https://www.npmjs.com/package/react-p5), which allows p5.js sketches to be mounted as React components!
- Backend: Node.js + Express and direct API call to LLM
    - We chose Node.js with Express for the backend because it is pretty light weight and well suited for building API endpoints for calling the Gemini API.
    - The backend will essentially receive inputs from the frontend, maintain and append to conversation history while also passing/calling the Gemini API with necessary prompts, and also parse the LLM response to extract the relevant code block.
    - To prevent issues regarding file size and type validations, we will limit this on both the frontend and backend.
- Data Flow
    - Input: LLM API: Gemini with Google AI Studio / Google Cloud Vertex AI
        - Text/natural language from the user, optional image upload (emotion/style reference), optional audio upload (voice narration/music)
    - Processing: frontend captures user inputs and converts it into tokens for the LLM, and the backend routes requests to API and returns structured output. LLM returns natural language response/explanation and p5.js code block.
    - Output: code and natural language output, rendered sketch from code, along with a chat message with explanation and comments

- Some considerations for the appdev layer:
    - Each conversation turn sends the full message history to maintain context, such that token usage grows with conversation length, which may not be cost effective.
    - To address this, we will only keep the last N turns to limit token costs and also condense the history such that large prompts aren't being constantly sent back and forth.
    - Other main costs would arise from the image/audio inputs, since these tend to be billed per image or per minute. To reduce token usage here, we will enforce limits that are clearly communicated to the user and integrated into the backend as well.

Product: Chatbot taking in natural language user input describing image they want with output being code in p5.js to represent that with image rendering of code